

ESM Credit LOS — Handover Guide (Maintainer README)

Audience: the next engineer taking over the ESM Credit (Loan Origination System) module.

Purpose: a living codebase guide so you can find your way, understand the architecture, and work safely without re-deriving everything from scratch.

Scope: the **Credit LOS module only** — backend/src/credit/ (backend), frontend/pages/credit/ + frontend/src/components/credit/ (frontend), and the credit-related Prisma models. The broader platform (Service Desk / HR / CRM / ITAM) is out of scope.

> Note: "ESM" appears twice in this repo. This document is about the **Credit LOS** module (the main project). There is also a small esm-workflow travel-request workflow (backend/src/controllers/esm-workflow.controller.ts, mounted at /api/v1/esm-workflow) which is **not** part of this handover.

1. What this module is

The Credit LOS module is a self-contained **loan origination / credit assessment system** embedded in the monolith. It takes a borrower through the full credit lifecycle:

```
borrower onboarding → application creation → KYC → underwriting →  
credit assessment → scoring/risk rating → committee approval →  
CA memo → conditions → offer/L00 → acceptance → disbursement → monitoring
```

It is effectively an **internal microservice approaching extraction point**: its own backend/src/credit/ directory with routes, controllers, services, middleware, queues, jobs, schedulers, validators, adapters, and templates.

Scale (verified counts)

Layer	Count
Route files	~74 (`backend/src/credit/routes/*.routes.ts`)
Controllers	69 (`backend/src/credit/controllers/`)
Service files	~108 (`backend/src/credit/services/*.ts`)
Dedicated middleware	14 (`backend/src/credit/middleware/`)
Credit Prisma models	~60+ (of 210 total in schema)
Frontend detail tabs	17 + sections (application workspace)

2. Architecture at a glance

Pattern: modular monolith, layered routes → controllers → services → Prisma, with a domain-driven module boundary around backend/src/credit/.

```
HTTP  
■■ backend/src/routes/index.ts ← mounts /api/v1/credit  
■■ backend/src/credit/routes/credit.routes.ts ← master credit router (all sub-routers)  
■■ sub-routers (per domain) backend/src/credit/routes/*.routes.ts  
■■ controllers backend/src/credit/controllers/*.controller.ts  
■■ services backend/src/credit/services/*.service.ts  
■■ middleware backend/src/credit/middleware/*.middleware.ts  
■■ Prisma backend/prisma/schema.prisma
```

Backend module structure (backend/src/credit/)

Directory	Contents
<code>`routes/`</code>	Express routers; <code>`credit.routes.ts`</code> is the master aggregator
<code>`controllers/`</code>	HTTP handlers, thin; delegate to services
<code>`services/`</code>	Business logic (~108 files, the meat)
<code>`middleware/`</code>	Cross-cutting: authz, RM scoping, feature flags, MFA, DLP, SOD, audit, encryption
<code>`adapters/`</code>	External integrations (bureau, AI, AV-scanner registry)
<code>`validators/`</code>	Joi request validation
<code>`queues/`</code>	BullMQ queue setup
<code>`jobs/`</code>	Recurring jobs: SLA checker, AML rescreen, collateral insurance monitor, audit retention, LOO expiry, monitor
<code>`schedulers/`</code>	Cron/node-cron schedulers (e.g. <code>`looExpiry.scheduler.ts`</code>)
<code>`templates/`</code>	Email/PDF templates
<code>`types/`, `constants/`, `utils/`</code>	Shared types, enums, helpers
<code>`__tests__`</code>	Unit tests (also per-dir <code>`__tests__`</code>)

How routes mount

- 1 `backend/src/routes/index.ts:95` → `router.use('/credit', creditRoutes)` → all credit endpoints live under `/api/v1/credit`.
- 2 `backend/src/credit/routes/credit.routes.ts` is the **master router**. It:
 - Serves a global **feature-flag gate**: `router.use(requireFeatureFlag('credit:module'))` (line 203) — the entire module is toggled by one flag.
 - Requires authenticate on everything (line 202), except a service-API-key AV-callback and the public feature-flags endpoint.
 - Applies **row-level access** once at the parent for all nested app resources:
`router.use('/applications/:applicationId', applyRmScope(), requireApplicationAccess())` (line 242).
 - Mounts each domain sub-router, often nested under `/applications` (facilities, parties, scoring, collateral, risks, ECL, monitoring, etc.).
 - Is grouped by **sprint/phase** in comments (Sprint 1..5, CA Memo Phases, § sections, P-* items) — the fastest way to understand history.

Frontend structure

- Routing in `frontend/App.tsx:308` — everything under `/credit` wrapped in `CreditLayout` and gated by `requirePermission="credit:read"`; child routes gate further (`credit:create`, `credit:approve`, `credit:admin`).
- Pages: `frontend/pages/credit/` (dashboard, application create, committee, group exposure, reports, mobile views).
- Application detail workspace: `frontend/pages/credit/tabs/` + `frontend/pages/credit/tabs/sections/` (17 tabs).

- Borrower360 workspace: frontend/src/components/credit/borrower360/.
- API client: frontend/src/services/credit.service.ts (~3,775 lines) — the single frontend service for the module.

3. Key cross-cutting controls (know these before you touch anything)

These are enforced per-request and will block or scope your work if you ignore them.

Control	Middleware	What it does
Module feature flag	<code>`requireFeatureFlag('credit:module')`</code> (<code>`featureFlag.middleware.ts`</code>)	Toggles the whole module; cached 60s
RM row scoping	<code>`applyRmScope()`</code> (<code>`rmScope.middleware.ts`</code>)	Non-admin users only see applications they are RM/analyst on; <code>`ADMIN/CREDIT_ADMIN/CREDIT_MANAGER`</code> and <code>`credit:admin`</code> bypass
Application row-level access	<code>`requireApplicationAccess()`</code> (<code>`applicationAccess.middleware.ts`</code>)	Re-asserts RM scope on direct-ID routes; out-of-scope reads return **404** (not 403 — avoids record existence disclosure)
Tiered RBAC	<code>`requirePermission(...)`</code> per action	<code>`credit:read/write/approve/disburse/admin`</code>
MFA	<code>`requireMfa`</code>	Required for admin mutations (e.g. feature flags) and approver/disburser roles (<code>`mfa.routes.ts`</code>)
Segregation of duties	<code>`sod.middleware.ts`</code> + server-side checks	e.g. recommendation author ≠ final decision actor; committee signers must differ
Auto-audit	<code>`autoAudit.middleware.ts`</code> , <code>`auditChain.service.ts`</code>	Audit events, immutable hash-chain
DLP / export control	<code>`dlp.middleware.ts`</code> , <code>`dlp.service.ts`</code>	Guards protected exports with tokens
PII read logging	<code>`piiReadLog.middleware.ts`</code>	Logs access to PII
Field encryption	<code>`fieldEncryption.middleware.ts`</code>	Encrypts sensitive fields at rest
Consent gating	<code>`consent.middleware.ts`</code>	PDPA consent enforcement

Permission surface: the module uses a fine-grained `credit:*` permission set (e.g. `credit:read`, `credit:write`, `credit:approve`, `credit:disburse`, `credit:admin`, plus per-domain `credit:borrowers`, `credit:collateral`, `credit:dlp`, `credit:str`, etc.).

4. The application state machine (the heart of the module)

The authoritative state machine lives in `backend/src/credit/services/creditApplication.service.ts` (the `TRANSITIONS` table, ~lines 178–248), not in the Prisma enum. `ApplicationState` is imported from `@prisma/client`.

19 states: `DRAFT`, `SUBMITTED`, `KYC_REVIEW`, `COMPLIANCE_HOLD`, `KYC_APPROVED`, `KYC_REJECTED`, `UNDERWRITING`, `CREDIT_ASSESSMENT`, `COMMITTEE_REVIEW`, `APPROVED`, `REJECTED`, `CONDITION_FULFILMENT`, `OFFER`, `ACCEPTED`, `DISBURSED`, `ACTIVE`, `CLOSED`, `WITHDRAWN`, `REFERRED_BACK`.

Terminal states (no outgoing transitions): `REJECTED`, `CLOSED`, `WITHDRAWN`.

Each transition maps to a required permission (`TRANSITION_PERMISSIONS`) and may require a reason and/or a structured rejection-reason code. Full detail and the gate list in [\[03-application-lifecycle.md\]\(03-application-lifecycle.md\)](#).

5. Working in this module — practical rules

- 1 **The state machine is the source of truth for transitions.** Do not bypass `transitionApplication()`; new transitions must be added to the `TRANSITIONS` table + `TRANSITION_PERMISSIONS` + `ACTION_LABELS`.
- 2 **Committee entry is heavily gated** (`enforceCommitteeEntryGate`). Adding a new committee requirement means touching `isCommitteeEntryAction` gates and `committeeEntryGate.ts`.
- 3 **Never break immutability invariants:** audit events are hash-chained; CA memo versions are immutable snapshots; post-decision document deletes are blocked (`stateGuard.util.ts`).
- 4 **Respect row scoping.** New list/detail routes that take an `applicationId` must be nested under the `/applications/:applicationId` parent guard (or re-apply `applyRmScope`).
- 5 **Feature-flag-gate new capabilities** (`credit:module` or a new sub-flag via `requireFeatureFlag`).
- 6 **Preserve the layering:** routes → controllers → services → Prisma. Controllers stay thin.
- 7 **Tests live beside code** (`__tests__`/ dirs). Run them before/after changes (`cd backend && npm test`).
- 8 **Soft deletes:** many entities have `deletedAt`; query with `deletedAt: null`.

6. Document index (this handover set)

Doc	Covers
00-README.md (this)	Landing page, architecture, controls, working rules
01-codebase-map.md	Directory-by-directory backend + frontend map
02-data-model.md	Credit Prisma models grouped by domain, enums, invariants
03-application-lifecycle.md	State machine, transitions, gates, RBAC per action
04-business-flows.md	End-to-end flows: onboarding → disbursement → monitoring
05-api-surface.md	Credit API surface: routes, controllers, authz
06-frontend-map.md	Frontend routes, pages, tabs, components, services

Pre-existing docs worth reading (not regenerated here)

- `docs/ESM_AUDIT.md` — full stack/domain audit
- `docs/ESM_ARCHITECTURE_PRODUCTION_READINESS_AUDIT.md` — production-readiness audit
- `docs/credit-los-audit-2026-08-08/` — 14-file deep audit (capability map, journeys, scoring, RBAC, test matrix, remediation backlog)
- `docs/audits/` — journey audits (application-creation, borrower-creation, workspace)
- `docs/remediation/` — 5-phase application-workspace remediation plan
- `docs/credit-scorecard-architecture.md`, `docs/CREDIT_ASSESSMENT_FSD_SDD.md`
- `docs/Executive_Transition_Pack_*.docx`

7. Quick start (dev)

```
# Backend
cd backend
npm install
cp .env.example .env # set DATABASE_URL, JWT secrets, CREDIT_ENCRYPTION_KEY
npm run prisma:generate
npx prisma migrate dev # apply migrations
npm run prisma:seed # seed accounts + credit reference data
npm run dev # tsx watch, port 3000

# Frontend
cd ../frontend
npm install
npm run dev # Vite, port 5173

# Tests
cd backend && npm test # or test:watch / test:coverage
```

Seeded accounts (passwords in .env): admin@test.local, it@test.local, hr@test.local, etc. Ensure a user with credit:read + appropriate credit roles is used to access /credit.

> The module is gated by the credit:module feature flag — if you get 403 "feature disabled", enable the flag for the tenant (PATCH /api/v1/credit/feature-flags/:key, requires credit:admin + MFA).